Hatchloom Inc
Design Document

Version:	1.8.4
Print Date:	
Release Date:	Mar 8, 2026
Release State:	
Approval State:	
Approved by:	
Prepared by:	
Reviewed by:	
Path Name:	
File Name:	
Document No:	

object oriented pty. ltd.

Document Change Control

Version	Date	Authors	Summary of Changes
1.0	Feb 24, 2026	Andrew Cao	Added purpose, overview, and some ConnectHub sequence diagrams
1.1	Feb 25, 2026	Andrew Cao	Added UML class diagrams for ConnectHub and design patterns used
1.2	Feb 26, 2026	Jinyang Zhong	- Added sequence diagram and details about user auth and profile - Added subheaders for each subpackage in Sequence Diagram section
1.3	Feb 27, 2026	Andrew Cao	Added example test cases for the Post creation and updating for ConnectHub service
1.4	Feb 28, 2026	Jinyang Zhong	- Added example test cases fo r register user, login with valid/invalid credentials, session validation, role check - Added Class diagrams for User, UserProfile, Session, Role
1.5	Feb 28, 2026	Andrew Cao	Added major design decisions and architecture patterns used
1.6	Feb 28, 2026	Ronald Kwok	Added Project Backlog and the Sprint Backlog Added GANTT Diagram Added Class Diagram for Launchpad
1.7	Mar 1, 2026	Jinyang Zhong	Updated class diagram to match new business requirements for roles and actors.
1.8.1	Mar 1, 2026	Anthony Toyco	Added 4.1 architecture component diagrams (System Overview, Auth and User Service, LaunchPad Service, ConnectHub Service)
1.8.2	Mar 1, 2026	Anthony Toyco	Added 4.2 interface specification in the architecture section
1.8.3	Mar 1, 2026	Anthony Toyco	Added sequence diagrams and descriptions for the LaunchPad home page, opening a SideHustle

Copyright Object Oriented Pty

			and Editing a BMC Section, and creating an open position on a SideHustle
1.8.4	Mar 1, 2026	Anthony Toyco	Added 4 more various test cases (User Registration, LaunchPad SideHustle creation, Business Model Canvas (BMC) update via API, and classified post state management)

Document Sign-Off

Name (Position)	Signature	Date

Copyright Object Oriented Pty

Contents

1 INTRODUCTION 5
1.1 Purpose 5
1.2 Overview 5
1.3 Resources - References 5
2 SEQUENCE DIAGRAMS 5
3 MAJOR DESIGN DECISIONS 5
4 ARCHITECTURE 5
4.1 Component Diagram 5
4.2 Interface Specification 5
4.3 Architectural Patterns Used 6
5 DETAILED CLASS DIAGRAMS 6
5.1 UML Class Diagrams 6
6 USE OF DESIGN PATTERNS 6
7 ACTIVITIES PLAN 6
7.1 Gantt Chart 6
7.1 Project Backlog and Sprint Backlog 6
7.2 Group Meeting Logs 6
8 TEST DRIVEN DEVELOPMENT 7

Copyright Object Oriented Pty

1 Introduction

1.1 Purpose

This document details the requirements of the system Hatchloom.

This project documents part of the necessary core backend implementation and design to power Hatchloom Inc, a learning platform for children

1.2 Overview

This document serves as the outline to solve the problems of implementing user authentication, session management, role-based access control, user profile management, a Launchpad service that includes sandbox environments and side hustles that the students want to pursue, and the ConnectHub service to integrate post creation and interactions such as liking and commenting a post, and adding messaging.

1.3 Resources - References

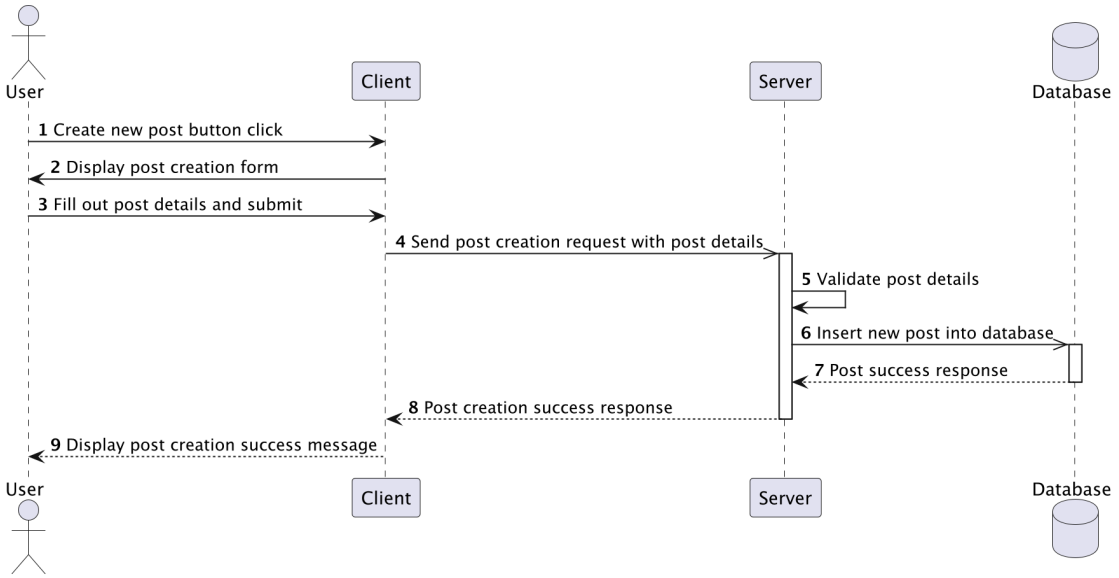
Include references to other documents that may assist in the understanding of this document. (i.e. past SRS and SDDs)

2 Sequence Diagrams

Here you will keep the Sequence Diagrams specified in Deliverable 1 unless you have made significant changes in your system.

Sequence Diagrams for the ConnectHub Service:

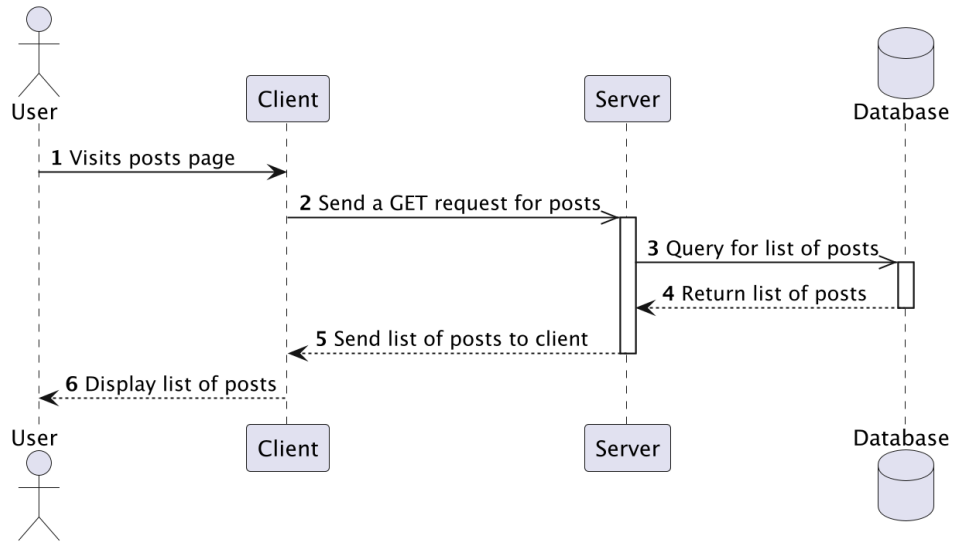
Creating a new post:



What this diagram represents is that it shows the process for a user creating a new post, for example sharing an achievement or announcement, similar to something like Linkedin. How it works is that the user will click the create post button, get shown the form and fill out the details and click the submit button, which will send a POST request to the Server, and the server will validate the information, and it will store in the database, and then return to the client with a 200 response status code.

Copyright Object Oriented Pty

Retrieving a list of posts when the user visits the page



This sequence diagram represents the process of retrieving a list of posts when the user visits the specific URL endpoint. When visiting the page, the client will automatically send a GET request to the Server, which will then have the database query for a list of posts to be returned back to the client and to be displayed on the frontend.

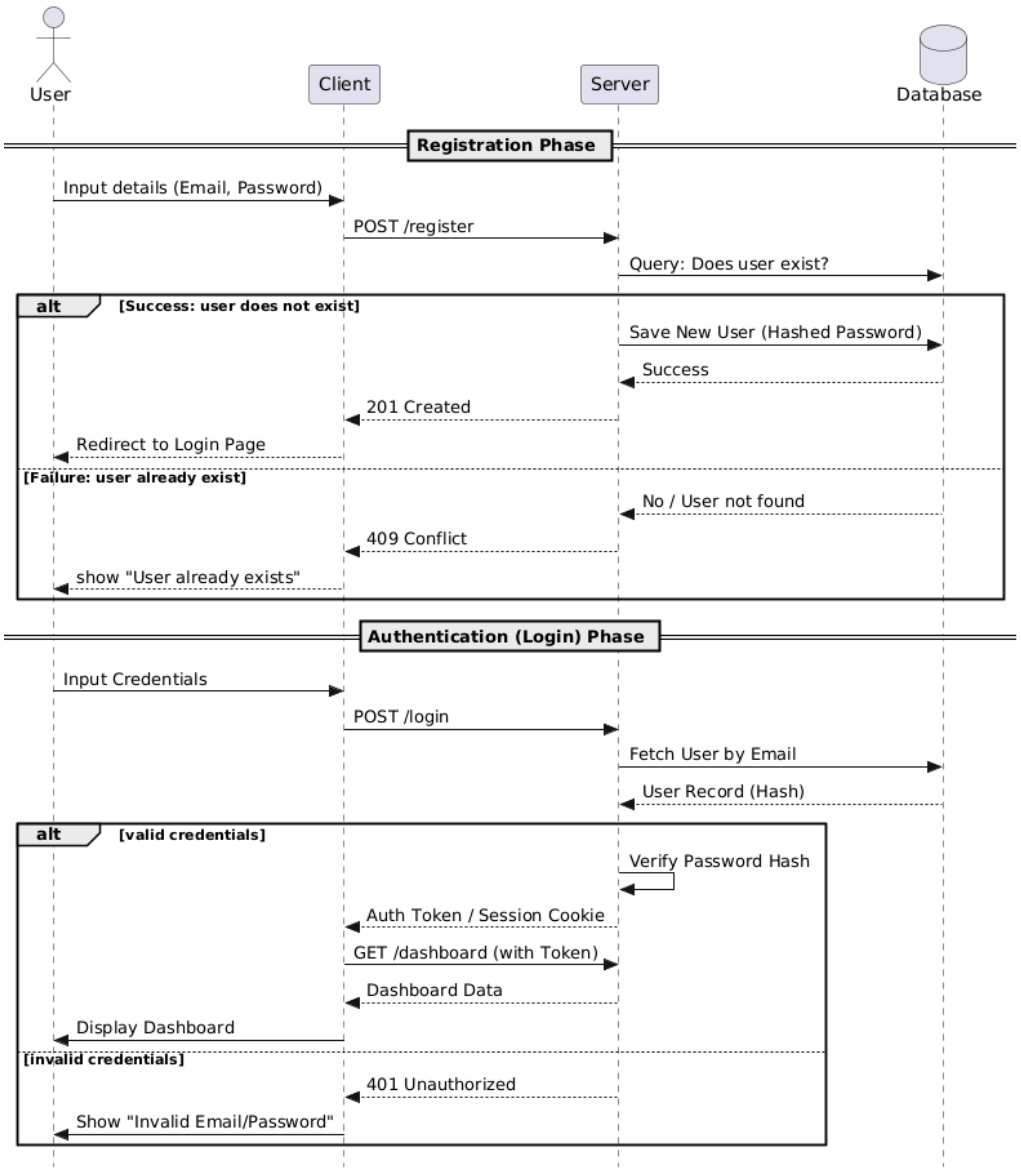
Creating a classified post and associating it with a project and status

This sequence diagram is similar to the regular post creation process, but instead with it being associated to a specific Launchpad Side hustle where again, the user clicks the classified post creation button, they are shown the form, and then they fill out the necessary information, along with the status, and the specific side hustle project and submit, to which the client will send a POST request to the server, the server will validate it, and insert it into the database linking with the specific project, and then the server will return a 200 status response back to the client.

Copyright Object Oriented Pty

Sequence Diagrams for user authentication and profile service:

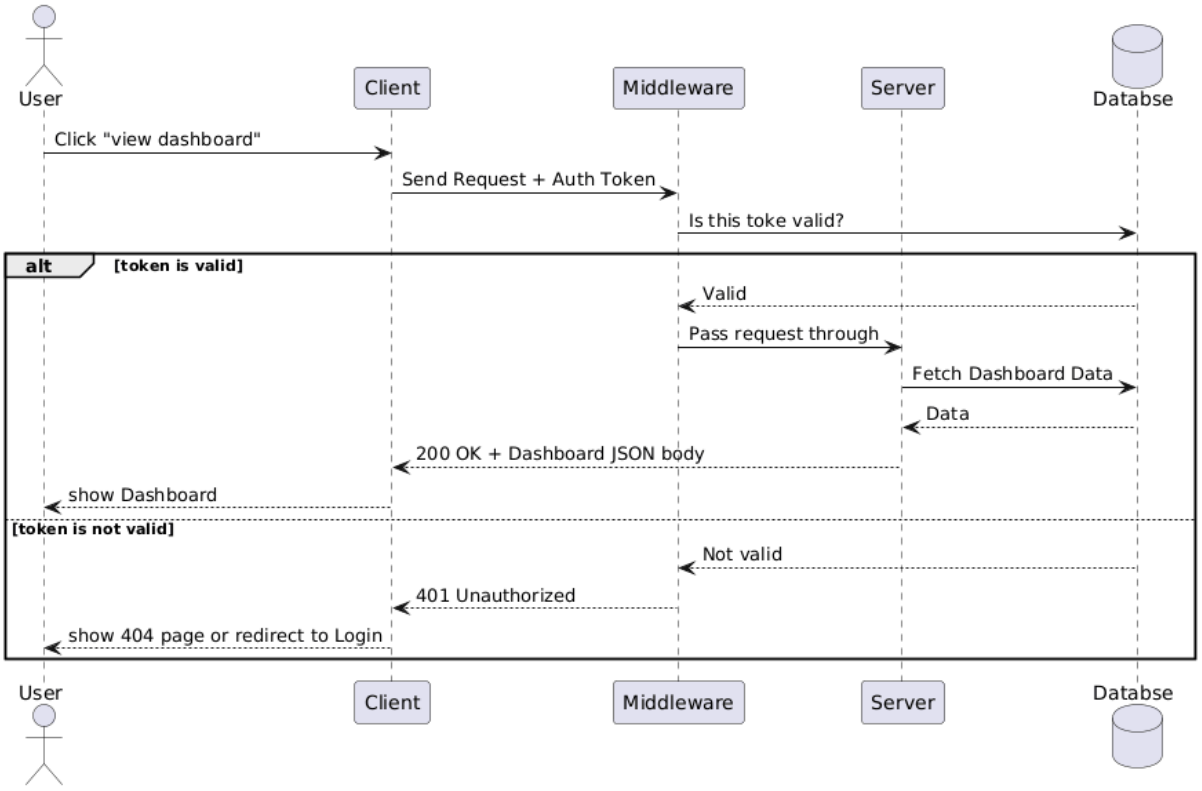
user registers, logs in, and is directed to dashboard



This sequence diagram captures the process of a user creating a new profile. The entire process can be split into 2 phases - registration and authentication. When a user creates an account, the server will first check if the given credentials collide with a column in the database, if not the account will be created and move on to authentication phase, else the server will return a 409 conflict status. Then when the user attempts to login, the server takes the request and validates the credentials, if successful it will redirect the user to the dashboard, if not it will display the failed login view.

Copyright Object Oriented Pty

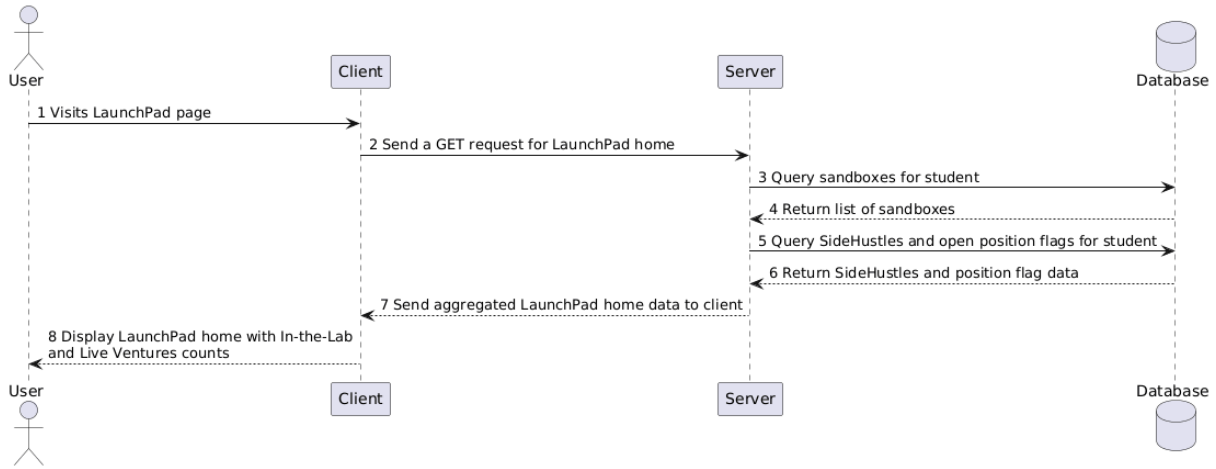
session validation on an API call (middleware pattern)



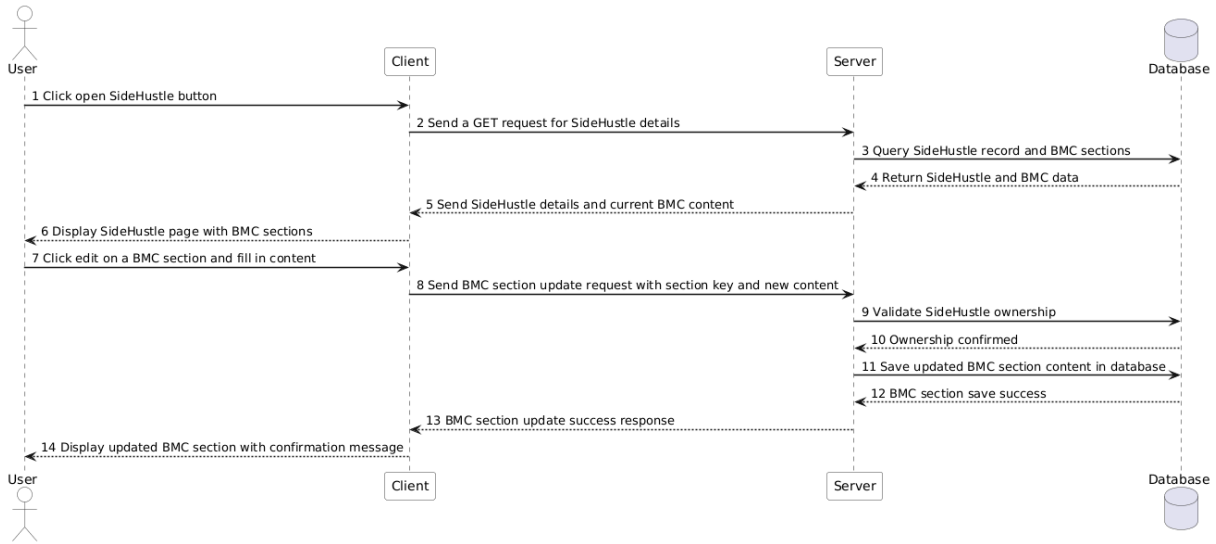
Here is the sequence diagram for session validation with requests. In this example the user is accessing the dashboard, but the logic applies on all requests to the server. First the client sends a request, then the middleware intercepts it and checks if the token is valid. If it is valid the request goes through to the server, if not the middle ware returns 401 unauthorized requests to the client and displays requests failed to the user and redirect them to Login to do the authorization process.

Copyright Object Oriented Pty

Sequence Diagrams for the LaunchPad Service:

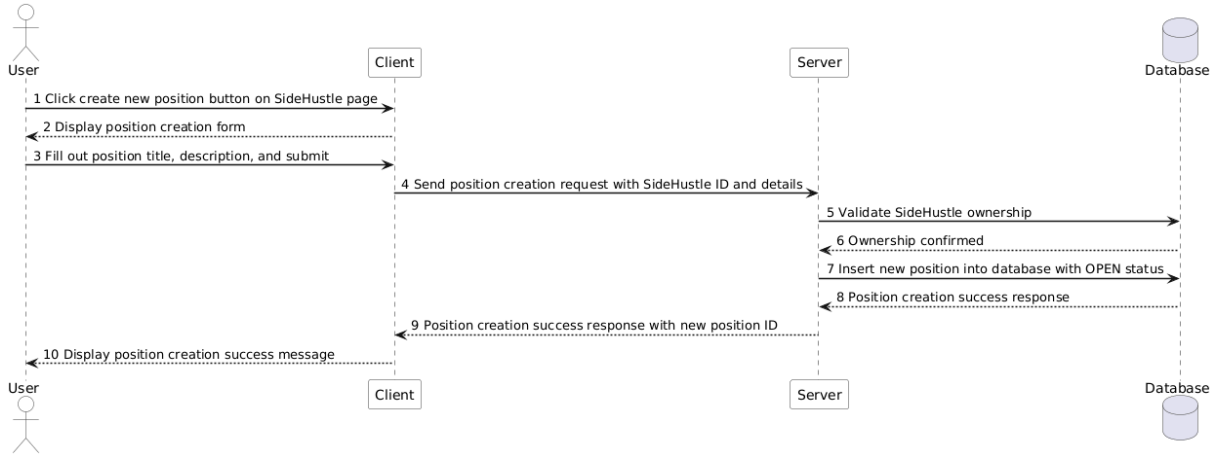


This sequence diagram represents the process of loading the LaunchPad home page when the student navigates to it. When visiting the page, the client automatically sends a GET request to the Server. The Server makes two queries to the database: one for the student's Sandboxes (In-the-Lab count) and one for the student's SideHustles along with any open-position flags (Live Ventures count). The aggregated home view is assembled by the Aggregator module and returned to the client, which then displays the LaunchPad overview to the student.



This sequence diagram represents the process of a student editing a section of their Business Model Canvas (BMC) within a SideHustle. The student first opens their SideHustle from the LaunchPad home page, triggering a GET request that retrieves the SideHustle record alongside all nine BMC sections. The client renders the SideHustle detail page with the current BMC content visible. When the student clicks to edit a specific BMC section (e.g. Value Propositions, Revenue Streams, Key Partners), they fill in the content and submit. The client sends a PUT request to the Server with the section key and updated text. The Server validates that the requesting student owns the SideHustle, then saves the updated section record to the database and returns a 200 success response. The client then reflects the change on the BMC section immediately.

Copyright Object Oriented Pty



This sequence diagram represents the process of a student creating a new open position on their SideHustle venture, similar to posting a job listing on a platform like LinkedIn. The student clicks the create position button on their SideHustle page and is shown the position creation form. They fill out the position title and description and click submit, which sends a POST request to the Server with the SideHustle ID and the position details. The Server validates that the requesting student is the owner of the SideHustle, then inserts the new position record into the database with an initial status of OPEN. The server returns a 200 success response to the client and the student sees a confirmation message. Once created, the open position becomes available to other students through the ConnectHub Classifieds module which reads the position status via the Position Status Interface.

3 Major Design Decisions

Description of significant design choices, and modularization criteria. Here you will keep and most probably extend the design decisions regarding the architecture of your system (i.e. the fact you have used a 3-tiered architecture because…). ***This directly relates with Section 4.3 below.***

For this application, for how complex, large, and for the amount of people it will support, it would be best to use a microservice architectural pattern rather than a monolithic architecture. If it was a monolithic architecture, it would be easier to develop, test and deploy, but this is mainly for applications that are smaller, and everything is more tightly coupled, so a lot of the code relies on one another. As for microservices, everything is a lot more organized and can scale independently by deploying more instances of that service, and more reliability as deploying changes for a specific service without bringing down the entire application.

Another design decision would be choosing an SQL database for our application, prioritizing strong consistency, keeping the data correct for things like user information, progress on side hustles or projects, and posts, rather than a NoSQL database which rather prioritizes availability.

Additionally, an architecture that will be used is REST due to being able to communicate with the server over HTTP using methods and a format like JSON to transfer and retrieve data, and that REST is simple to implement, and a lot of the

Copyright Object Oriented Pty

operations that will be done in this application are commonly CRUD, (GET, POST, PUT, DELETE).

Separation of concerns for splitting the frontend UI from the backend, where UI changes do not affect any business logic from the backend at all, and any devices using the application will use the same APIs to communicate with the server. Another thing is in our backend

Furthermore, another backend decision is using a layered architecture through a controller where it handles the routes, service where it handles all the logic, repository for data access, and then the database, for single responsibility pattern as the code is divided into 3 different parts where each class only has one reason to change.

Interface segregation principle can be used to split services into their very own specific ones, so instead of having a huge user interface for a service that can do things like create a user, delete a user, enroll them, etc, we can split it up into a user management interface and an enrollment interface for a UserManagementService and an EnrollmentService, so clients only depend on what they use.

4 Architecture

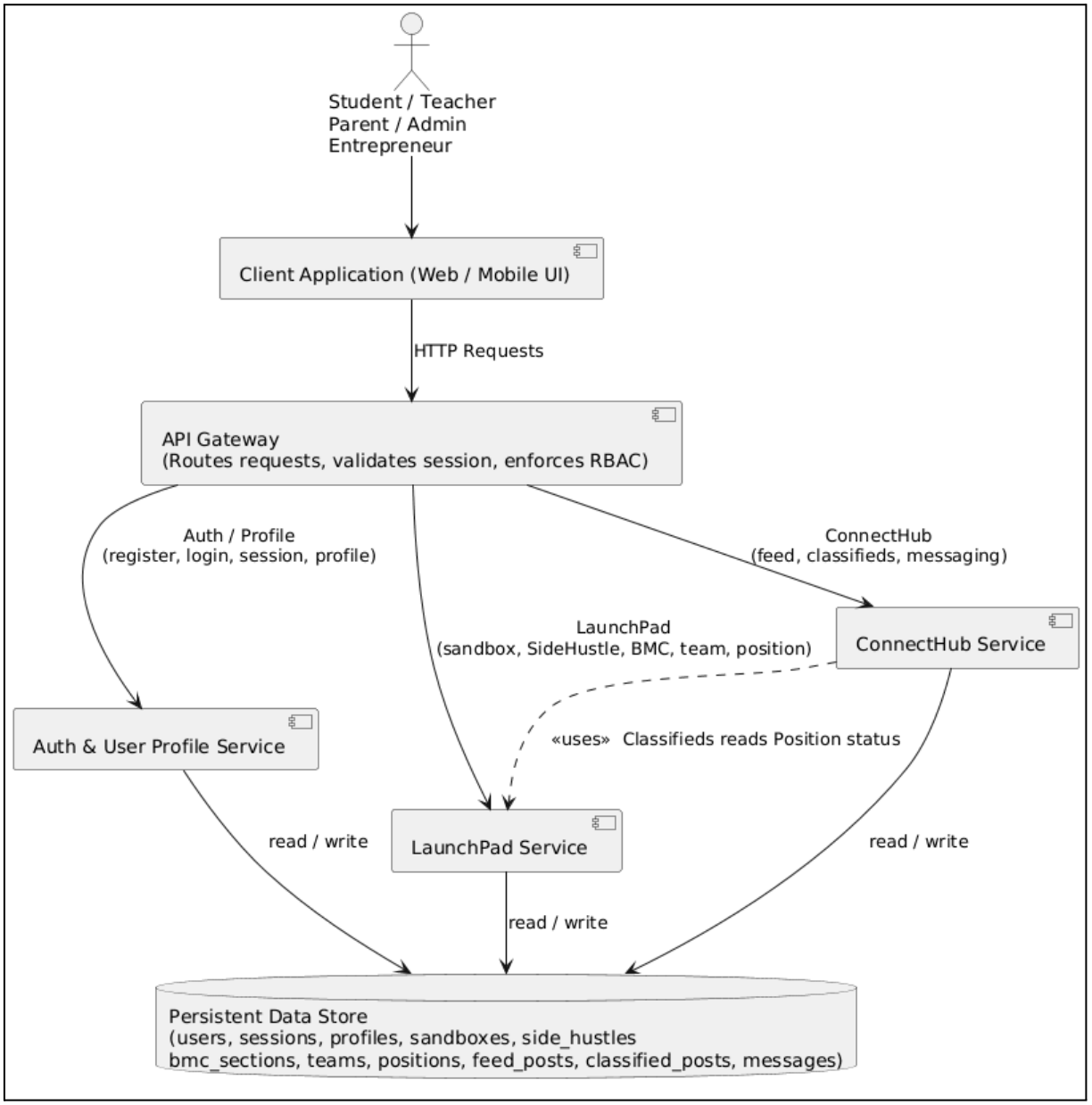
Provide a detailed up-to-date architecture specification of your system as it stands now. More specifically you will provide:

4.1 Component Diagram

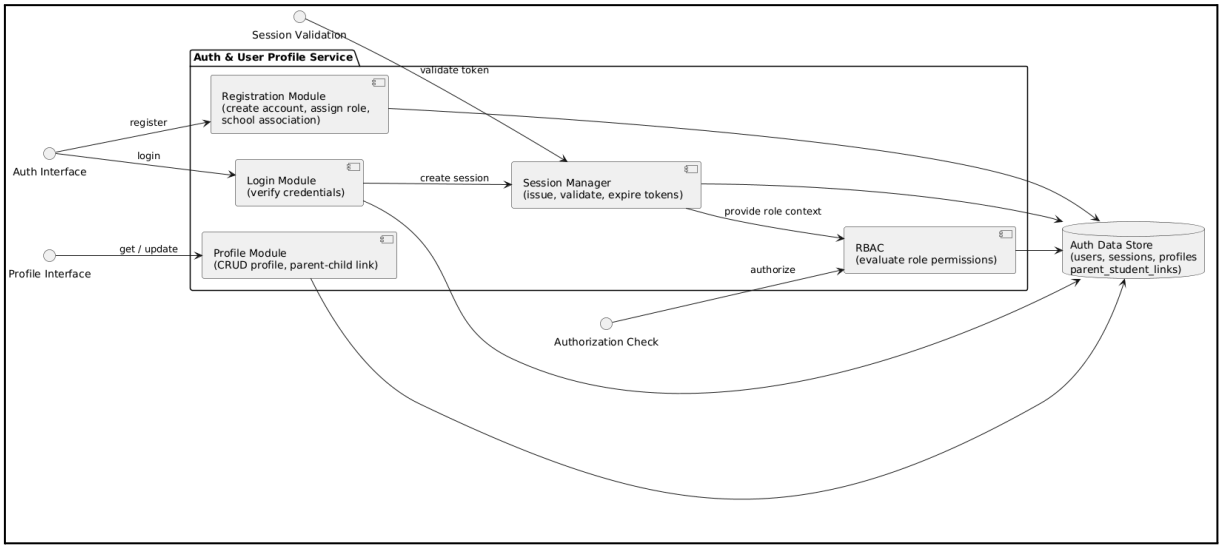
1. *A component diagram of your system clearly labelling each exposed interface. You may want to use more than one component diagrams in case of complex nested components so that you increase clarity and readability of your diagrams. Make sure you clearly show the client components (i.e. the components using the exposed interfaces as per UML Component diagram notation).*

System Overview:

Copyright Object Oriented Pty

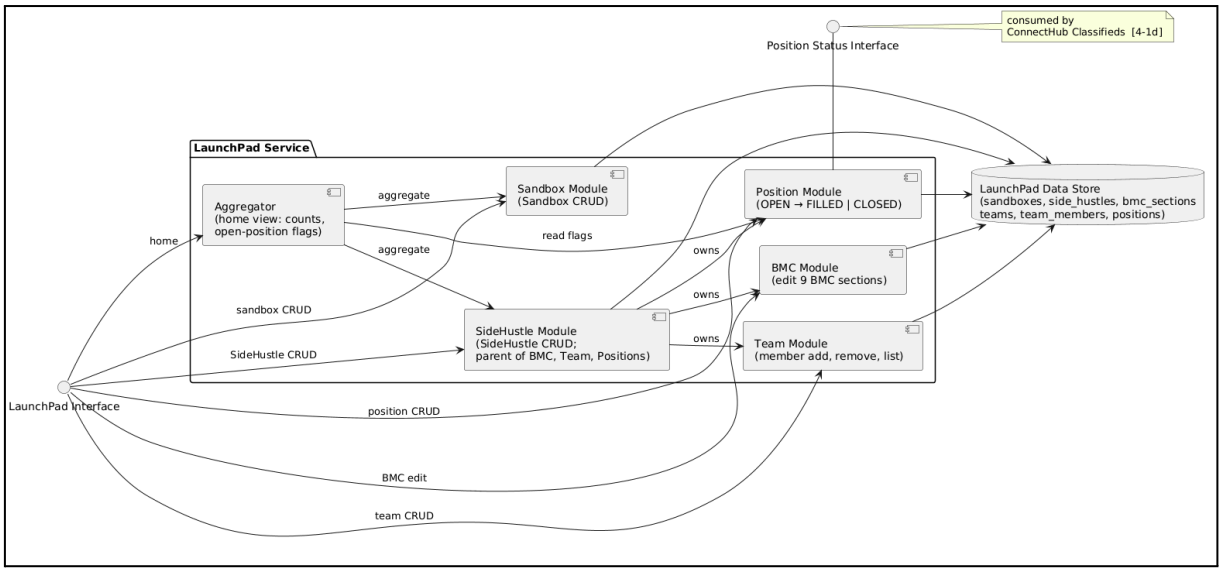


Auth & User Profile Service:

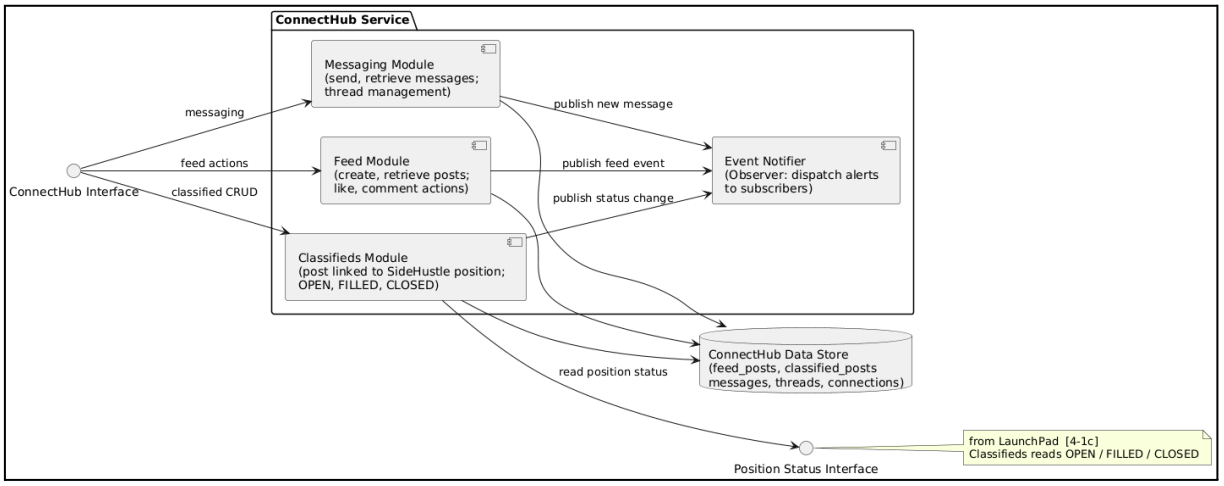


Copyright Object Oriented Pty

LaunchPad Service:



ConnectHub Service:



2. *A table with a description of each component as per the table below.*

Component Name	*Description*
Client Application	*The web or mobile UI through which all users (Student, Teacher, Parent, School Admin, Platform Admin, Entrepreneur) interact with the Hatchloom platform. All requests are sent exclusively through the API Gateway.*
API Gateway	*Central entry point for all backend requests. Routes each request to the correct service, validates the caller's session token before any business logic runs, and enforces role-based access control (RBAC) on every inbound call.*

Copyright Object Oriented Pty

Auth & User Profile Service	*Backend service responsible for user identity, authentication, session lifecycle, and profile management. No other service can function without a valid session token issued by this service.*
LaunchPad Service	*Backend service that manages the LaunchPad pillar: Sandbox workspaces, SideHustle ventures, Business Model Canvas data, team membership, and open positions.*
ConnectHub Service	*Backend service that manages the ConnectHub pillar: the social feed, classified job postings, and student messaging.*
Persistent Data Store	*Single logical database that stores all structured data for the platform, including users, sessions, profiles, sandboxes, side hustles, BMC sections, teams, positions, feed posts, classified posts, messages, and threads.*
Registration Module	*Creates a new user account with a username, hashed password, assigned role (e.g. Student, Teacher), and optional school association.*
Login Module	*Verifies submitted credentials against stored records. On success, delegates to the Session Manager to issue a session token.*
Session Manager	*Issues session tokens on login, validates tokens on every inbound request, and enforces token expiry. Provides role context to RBAC after validation.*
RBAC	*Evaluates whether the authenticated caller's role permits the requested operation. Applies a different permission strategy per role using the Strategy pattern.*
Profile Module	*Handles get and update operations on user profiles, manages parent–child account links, and provides an admin-only endpoint to list all profiles.*

Copyright Object Oriented Pty

Auth Data Store	*Scoped data store for the Auth service. Holds the users, sessions, profiles, and parent_student_links tables.*
Aggregator	*Composes the LaunchPad Home view by reading from Sandbox Module, SideHustle Module, and Position Module to produce In-the-Lab counts, Live Ventures counts, and open-position flags. Does not persist data itself.*
Sandbox Module	*Provides full CRUD operations for Sandbox workspaces owned by a student.*
SideHustle Module	*Provides full CRUD operations for SideHustle ventures. Acts as the parent owner of the BMC, Team, and Position sub-resources scoped to each venture.*
BMC Module	*Allows a student to edit and save each of the nine standard Business Model Canvas sections (Key Partners, Key Activities, Key Resources, Value Propositions, Customer Relationships, Channels, Customer Segments, Cost Structure, Revenue Streams) for a given SideHustle.*
Team Module	*Manages team membership within a SideHustle: add member, remove member, and list current members.*
Position Module	*Creates open roles within a SideHustle and manages the position status lifecycle (OPEN -> FILLED or OPEN -> CLOSED). Exposes the Position Status Interface consumed by the ConnectHub Classifieds Module.*
LaunchPad Data Store	*Scoped data store for the LaunchPad service. Holds the sandboxes, side_hustles, bmc_sections, teams, team_members, and positions tables.*
Feed Module	*Creates and retrieves feed items of types share, announcement, and achievement. Handles social actions (like, comment) on posts.*
Classifieds Module	*Creates classified job posts linked to a SideHustle position. Manages post status (OPEN, FILLED, CLOSED) and reads current position status from*

Copyright Object Oriented Pty

	the LaunchPad Position Status Interface to keep the two in sync.
Messaging Module	*Sends and retrieves messages and threads between users. Uses a context_type + context_id field pattern to associate messages with any platform context (e.g. a SideHustle or challenge).*
Event Notifier	*Implements the Observer pattern. Receives publish events from Feed Module, Classifieds Module, and Messaging Module, then dispatches alerts and notifications to subscribed users or UI widgets.*
ConnectHub Data Store	*Scoped data store for the ConnectHub service. Holds the feed_posts, classified_posts, messages, threads, and connections tables.*

4.2 Interface Specification

1. *A table listing the signatures of the operations contained in each exposed interface of your component diagram as per the table below.*

Interface Name	*Signature of Operations Exposed in Interface*	*Description of the Operation*
Auth Interface	*void register(String username, String password, String role, String schoolId)*	*Creates a new user account with a hashed password, an assigned role (e.g. Student, Teacher, Entrepreneur), and an optional school association. Delegates to the Registration Module.*
	String login(String username, String password)	*Verifies the submitted credentials against stored records. On success, delegates to the Session Manager to create and return a session token.*
Profile Interface	*UserProfile getProfile(String userId)*	*Retrieves the full profile record for the specified user ID, including role, school association, and any parent–child links.*

Copyright Object Oriented Pty

	void updateProfile(String userId, UserProfile updates)	*Applies partial or full updates to the profile fields of the specified user (e.g. display name, bio, school).*
	List<UserProfile> listProfiles()	*Admin-only operation. Returns the complete list of user profiles registered on the platform.*
Session Validation Interface	*SessionContext validateToken(String token)*	*Validates the given session token, checks expiry, and returns a SessionContext object containing the user ID and role. Throws an exception or returns null if the token is invalid or expired.*
Authorization Check Interface	*boolean authorize(String token, String resource, String action)*	*Evaluates whether the role associated with the session token is permitted to perform the specified action on the given resource. Applies the role's Strategy-pattern permission rules. Returns true if permitted, false otherwise.*
LaunchPad Interface	*LaunchPadHome getHome(String userId)*	*Returns the aggregated LaunchPad Home view for the student: In-the-Lab count (active Sandboxes), Live Ventures count (active SideHustles), and open-position flags. Composed by the Aggregator without persisting data.*
	Sandbox createSandbox(String userId, String title, String description)	*Creates a new Sandbox workspace owned by the specified student. Returns the newly created Sandbox object with its generated ID.*

Copyright Object Oriented Pty

	Sandbox getSandbox(String sandboxId)	*Retrieves a Sandbox workspace by its ID.*
	void updateSandbox(String sandboxId, String title, String description)	*Updates the title or description of an existing Sandbox workspace.*
	void deleteSandbox(String sandboxId)	*Permanently removes a Sandbox workspace from the data store.*
	SideHustle createSideHustle(String userId, String title)	*Creates a new SideHustle venture owned by the specified student. Returns the newly created SideHustle object. Acts as the parent owner of BMC, Team, and Position sub-resources.*
	SideHustle getSideHustle(String sideHustleId)	*Retrieves a SideHustle venture by its ID, including summary information about its associated BMC, team, and positions.*
	void updateSideHustle(String sideHustleId, SideHustle updates)	*Updates the metadata (e.g. title, description, status) of an existing SideHustle.*
	void deleteSideHustle(String sideHustleId)	*Deletes a SideHustle and all of its owned sub-resources (BMC sections, team members, positions).*
	BMCData editBMC(String sideHustleId, String section, String content)	*Saves the content for one of the nine standard Business Model Canvas sections (e.g. Key Partners, Value Propositions, Revenue Streams) for the given SideHustle. Returns the updated BMCData record.*
	void addTeamMember(String	*Adds a user to the team of a specified SideHustle venture.*

Copyright Object Oriented Pty

	sideHustleId, String userId)	
	void removeTeamMember(String sideHustleId, String userId)	*Removes a user from the team of a specified SideHustle venture.*
	List<User> listTeamMembers(String sideHustleId)	*Returns the current list of team members for a specified SideHustle.*
	Position createPosition(String sideHustleId, String title, String description)	*Creates a new open role (OPEN status) within a SideHustle. Returns the newly created Position object with its generated ID.*
	void updatePositionStatus(String positionId, String status)	*Transitions the lifecycle status of a position. Valid transitions are OPEN -> FILLED and OPEN -> CLOSED.*
Position Status Interface	*String getPositionStatus(String positionId)*	*Returns the current status (OPEN, FILLED, or CLOSED) of the specified position. This interface is consumed by the ConnectHub Classifieds Module to keep classified post status in sync with the source position.*
ConnectHub Interface	*FeedPost createFeedPost(String userId, String type, String content)*	*Creates a new feed post of the specified type (share, announcement, or achievement) authored by the given user. Triggers the Event Notifier to dispatch alerts to subscribers.*
	List<FeedPost> getFeed(String userId)	*Retrieves the social feed visible to the specified user, ordered by recency.*
	void likePost(String postId, String userId)	*Records a like action by the specified user on the given feed post.*
	void commentOnPost(String	*Appends a comment authored by the specified*

Copyright Object Oriented Pty

	postId, String userId, String comment)	*user to the given feed post.*
	ClassifiedPost createClassifiedPost(String positionId, String title, String description)	*Creates a classified job post linked to the specified SideHustle position. Initial status is set to OPEN. Triggers the Event Notifier on creation.*
	List<ClassifiedPost> listClassifieds(String status)	*Returns classified job posts, optionally filtered by status (OPEN, FILLED, or CLOSED).*
	void updateClassifiedStatus(String postId, String status)	*Updates the status of a classified post to mirror the lifecycle of its linked SideHustle position (e.g. OPEN -> FILLED when the position is filled). Triggers the Event Notifier on status change.*
	Message sendMessage(String senderId, String receiverId, String content, String contextType, String contextId)	*Sends a message from one user to another. Associates the message with a platform context (e.g. a SideHustle or challenge) via the contextType and contextId fields. Triggers the Event Notifier to alert the recipient.*
	List<Message> getMessages(String threadId)	*Retrieves all messages within the specified thread, ordered chronologically.*

4.3 Architectural Patterns Used

A discussion (two paragraphs in total) of the architectural patterns used, and the rationale behind your choice.

One of the architectural patterns used in this application would be REST, as described earlier, it allows us to communicate with the server using HTTP methods and allow us to transfer and retrieve data formats like JSON where it is a lot simpler, easier to handle and extract, and also that REST as an architectural pattern is simple to implement, and that majority of the operations done in this application will be CRUD (GET, POST, PUT, DELETE). Another architectural pattern we can use is the Layered architecture, which is quite similar to the version of MVC, but instead, we use a

Copyright Object Oriented Pty

Controller to handle the routes or requests, Services to handle all the business logic, Repository to handle data access and interactivity, and then there is the database. This 3-4 layer architecture promotes the single responsibility pattern, where dividing the code into different layers ensures that each layer focuses on one specific thing inside the backend.

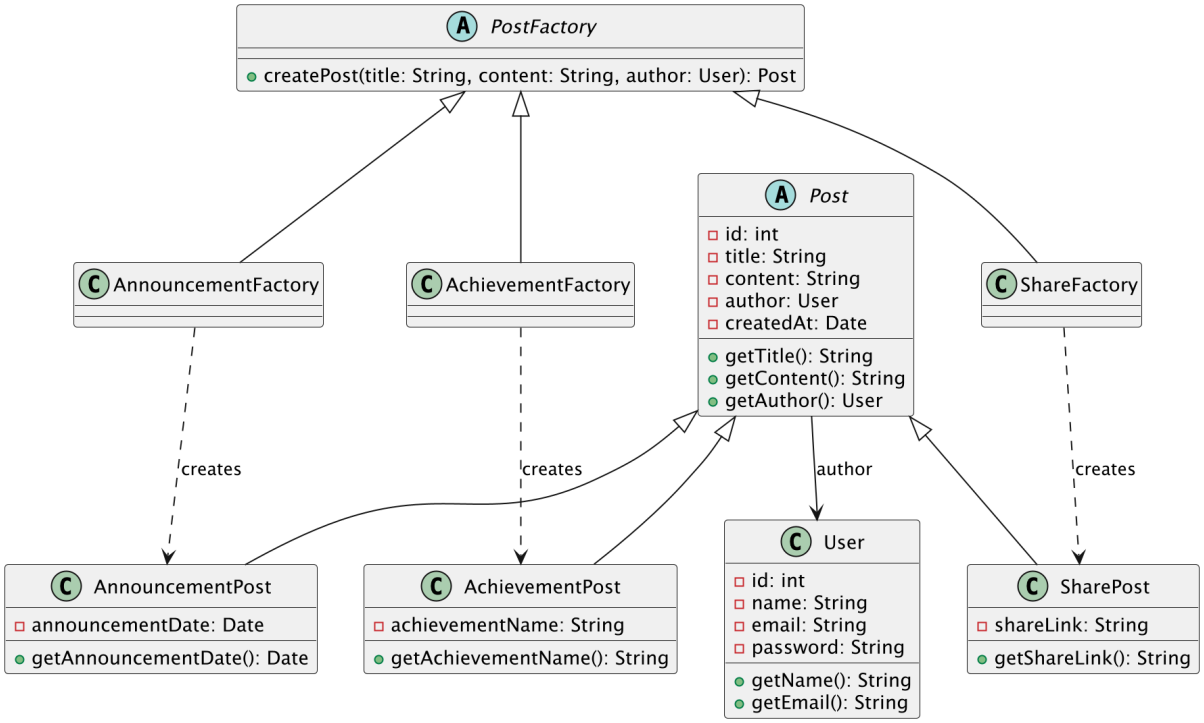
Furthermore, Multitier architecture here is basically an extension to the REST and Client-server architecture. This is a web application that has more than two entities, with there being a client where it is just a graphical interface that they make requests from, and then the server will respond, but with an extra level which is the database, that the server will use to interact with. This is a fundamental architecture of the internet and is really generic as pretty much all web applications use this architecture.

5 Detailed Class Diagrams

5.1 UML Class Diagrams

Detailed class diagrams for each component in your system. It is advised that you use a tool like ObjectAid to automatically create the diagrams from your code and not draw them manually. You can have a separate class diagram per component (i.e. Java package) ObjectAid has been tried with Eclipse and Java 8. You can dowload ObjectAid from http://web.archive.org/web/20190113065017/http://www.objectaid.com/download

Instructions on how to install and run it on Eclipse can be seen at https://www.youtube.com/watch?v=UtSRX9swc1w



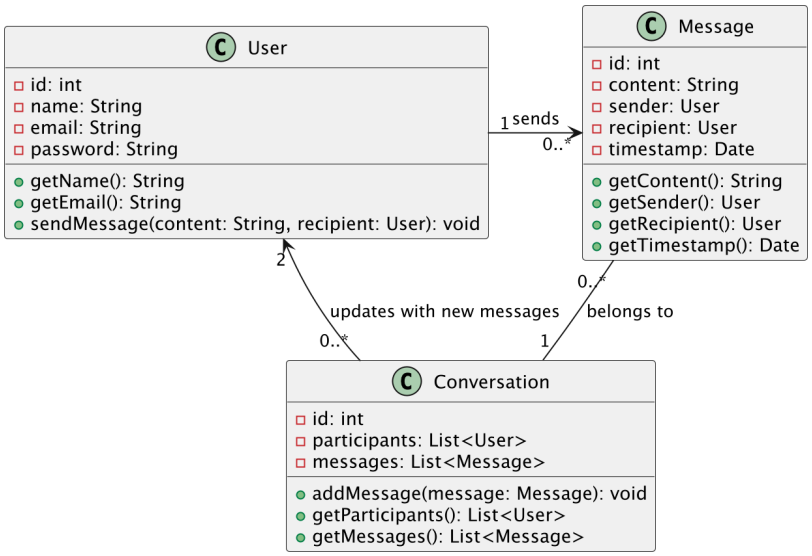
Class name	*Attribute/Method name*	*Description*

Copyright Object Oriented Pty

PostFactory	*createPost(title: String, content: String, author: User): Post*	*An abstract factory creator for the different post types that subclasses will implement for new types*
ShareFactory	*createPost(title: String, content: String, author: User): Post*	*A subclass creator that returns a post of type Share*
AnnouncementFactory	*createPost(title: String, content: String, author: User): Post*	*A subclass creator that returns a post of type Announcement*
AchievementFactory	*createPost(title: String, content: String, author: User): Post*	*A subclass creator that returns a post of type Achievement*
Post	*id: int* *title: String* *content: String* *author: User* *createdAt: Date*	*The unique identifier for the post* *The title for the specific post* *The description/content bulk for the post* *The poster* *Date the post was created*
Post	*getTitle(): String*	*Get the title for the specific post*
Post	*getContent(): String*	*Get the description/content bulk for the post*
Post	*getAuthor(): User*	*Get the user who created the post*
User	*id: int* *name: String* *email: String* *password: String*	*Unique identifier for the User* *Name for the student* *Email for the student* *Password for the student*
User	*getName(): String*	*Get the student's name*
User	*getEmail(): String*	*Get the student's email*
AnnouncementPost	*announcementDate: Date* *getAnnouncementDate():*	*Date for the announcement post*

Copyright Object Oriented Pty

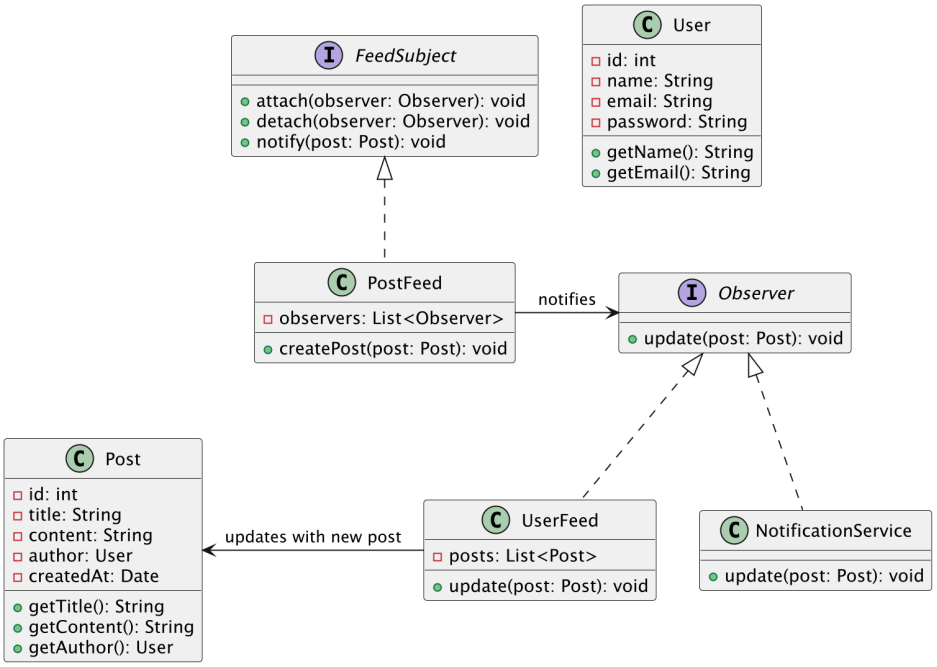
	Date	*Get the announcement date for the post*
AchievementPost	*achievementName: String* *getAchievementName: String*	*Name for the achievement earned* *Get the name of the achievement earned*
SharePost	*shareLink: String* *getShareLink(): String*	*Unique identifier link for the post to be shared* *Get the unique identifier link for the post to be shared*



Class	*Attribute/Method name*	*Description*
User	*id: int* *name: String* *email: String* *password: String*	*Unique identifier for the user* *Name for the student* *Email for the student* *Password for the student*
User	*getName(): String* *getEmail(): String* *sendMessage(content: String, recipient: User): void*	*Get the student's name* *Get the student's email* *Sends a message with the text to the intended receiver*

Copyright Object Oriented Pty

Message	*id: int* *content: String* *sender: User* *recipient: User* *timestamp: Date*	*Unique identifier for the message* *Text to be sent in the message* *The person who sent the message* *The person who will receive the message* *The time the message was sent*
Message	*getContent(): String* *getSender(): User* *getRecipient(): User* *getTimestamp(): Date*	*Getting the text sent from the message* *Getting the person who sent the message* *Getting the person who received the message* *Getting the time for when the message was sent*



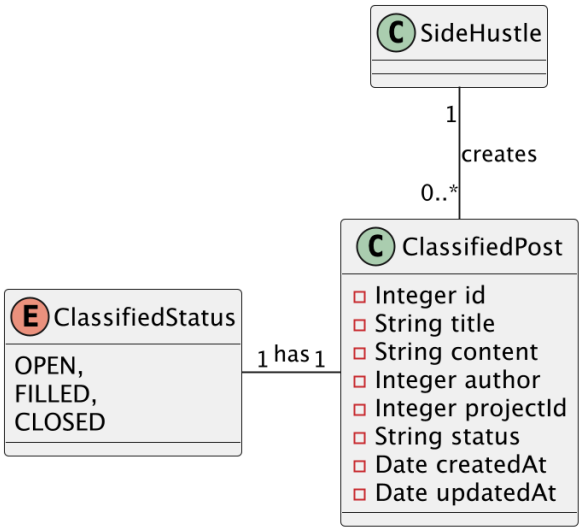
Class	*Attribute/method name*	*Description*

Copyright Object Oriented Pty

User	*id: int* *name: String* *email: String* *password: String*	*Unique identifier for the user* *Name for the student* *Email for the student* *Password for the student*
User	*getName(): String* *getEmail(): String* *sendMessage(content: String, recipient: User): void*	*Get the student's name* *Get the student's email* *Sends a message with the text to the intended receiver*
Post	*id: int* *title: String* *content: String* *author: User* *createdAt: Date*	*The unique identifier for the post* *The title for the specific post* *The description/content bulk for the post* *The poster* *Date the post was created*
Post	*getTitle(): String*	*Get the title for the specific post*
Post	*getContent(): String*	*Get the description/content bulk for the post*
Post	*getAuthor(): User*	*Get the user who created the post*
Observer	*update(post: Post): void*	*An interface for the observer that provides an update method*
FeedSubject	*attach(observer: Observer): void* *detach(observer: Observer): void* *notify(post: Post): void*	*An interface for managing the observers (attaching and detaching) for the feed, and a notify method for sending a notification that new changes were made*
PostFeed	*observers: List<Observer>* *createPost(post: Post): void*	*Maintain a collection of observers that were attached* *And will have a createPost method that*

Copyright Object Oriented Pty

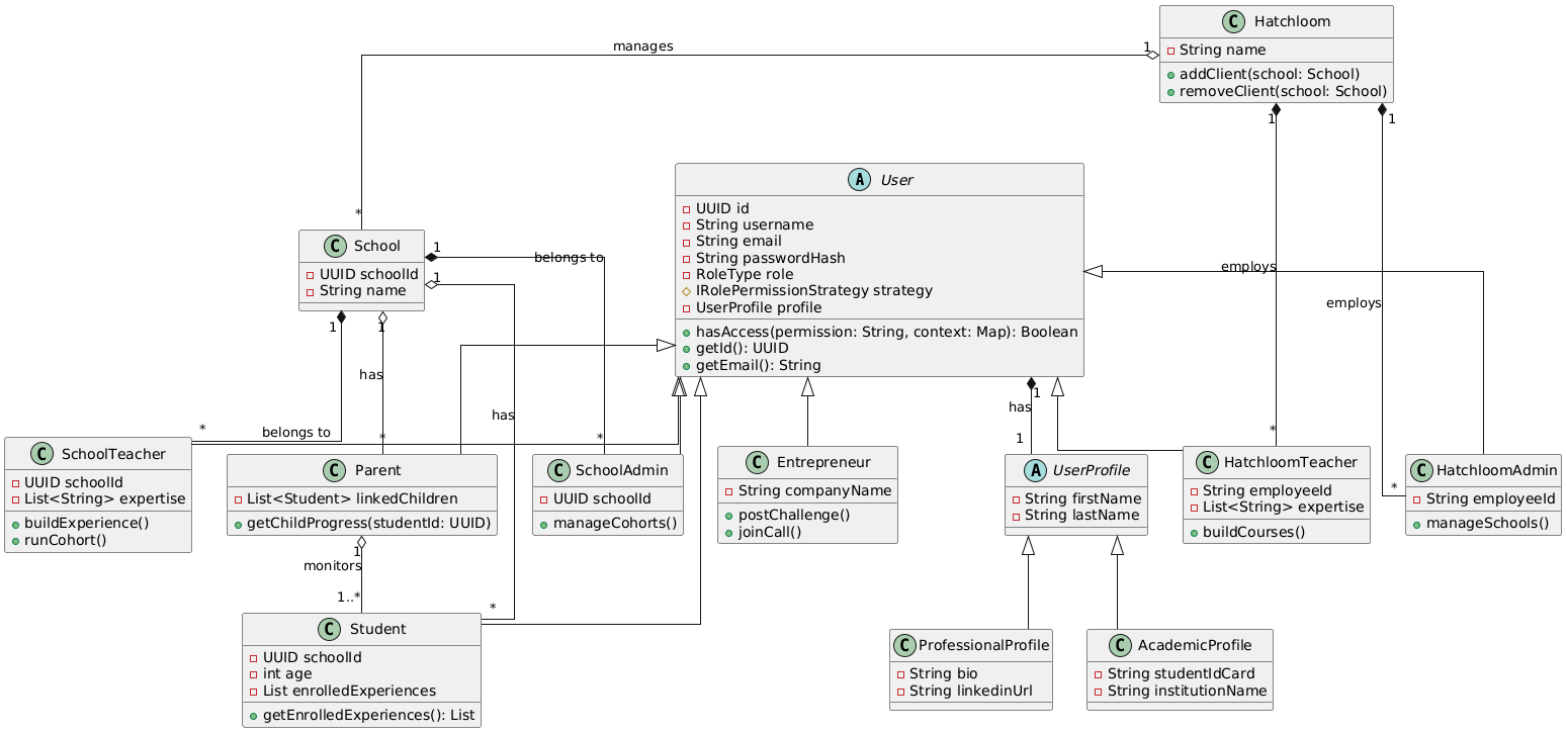
		will be used to notify the observers
UserFeed	*posts: List<Post>* *update(post: Post): void*	*Maintain the list of posts that will be displayed back to the user feed, and update method will add the new post to the feed*
NotificationService	*update(post: Post): void*	*Sends the notification that a new post was added*



Class	*Attribute/method name*	*Description*
SideHustle	*Refer to launchpad service*	*Side hustle is a project from the launchpad service that will be calling the ConnectHub microservice for creating a classified post*
ClassifiedStatus	*OPEN, FILLED, CLOSED*	*Contains the current status of the classified post*
ClassifiedPost	*Id: Integer* *title: String* *author: Integer* *projectId: Integer* *status: String*	*Contains the classified post Id to identify, the title of the post, author which is who created the post, projectId, the id that*

Copyright Object Oriented Pty

	createdAt: Date *updatedAt: Date*	*the project is associated with that, Status for the post's current status, and dates for when the post was created and updated.*



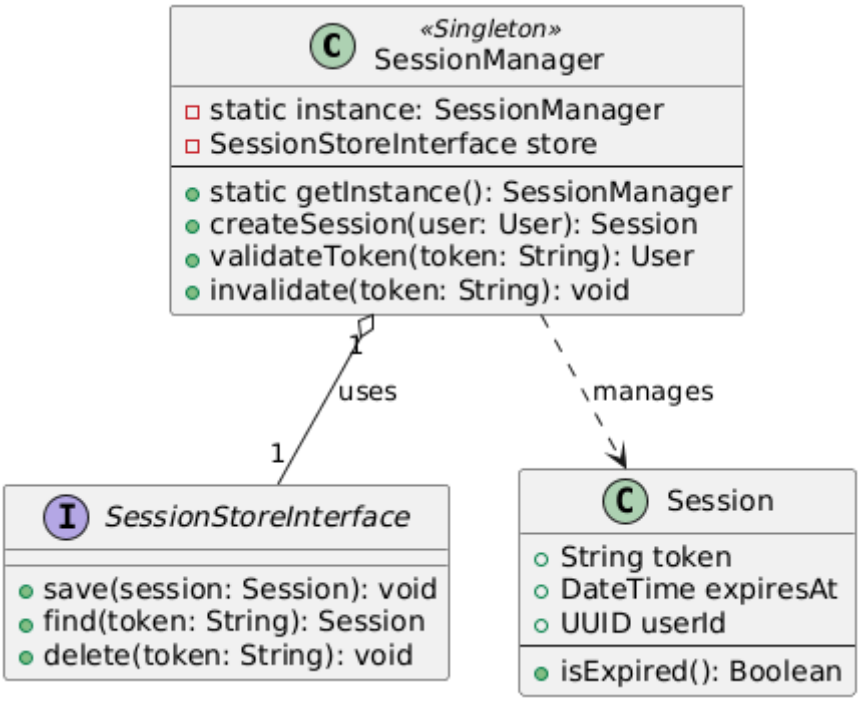
Class	*Attributes*	*Methods*	*Description*
User	*UUID id, String username, String email, String passwordHash, RoleType role, IRolePermissionStrategy strategy, UserProfile profile*	*hasAccess(permission: String, context: Map): Boolean, getId(): UUID, getEmail(): String*	*The identity layer stores core authentication and role data. `hasAccess` delegates logic to the role strategy.*
UserProfile	*String firstName, String lastName*		*The profile layer holds non-sensitive information for display purposes.*
Student	*UUID schoolId, int age, List enrolledExperiences*	*getEnrolledExperiences(): List*	*Primary user. Tied to a School. This class is the "whole" composition within*

Copyright Object Oriented Pty

			the Parent class.
Parent	*linkedChildren: ArrayList<Student>*	*getChildProgress(studentId: UUID)*	*Dependent account of the Student class. Exists only as a part of a student's lifecycle. Must be linked to a student.*
HatchloomTeacher	*String employeeId, List<String> expertise*	*buildCourses()*	*Hatchloom Employee. Teachers can build courses.*
HatchloomAdmin	*String employeeId*	*manageSchools()*	*Scoped Admin. Read-only access restricted strictly to their assigned schoolId*
Entrepreneur	*companyName: String*	*postChallenge(), joinCall()*	*Guest Role. Limited to posting challenges and joining calls.*
Platform Admin	*employeeId: String*		*System Admin. Full unrestricted access for onboarding and moderation.*
AcademicProfile	*String studentIdCard, String institutionName*		*Specialized profile for Students and Teachers.*
ProfessionalProfile	*String bio, String linkedinUrl*		*Specialized profile for Entrepreneurs and Admins.*
Hatchloom	*String name*	*addClient(school: School), removeClient(school: School)*	*Root organizational entity that manages School clients and employs platform-level staff.*
SchoolTeacher	*UUID schoolId, List expertise*	*buildExperience(), runCohort()*	

Copyright Object Oriented Pty

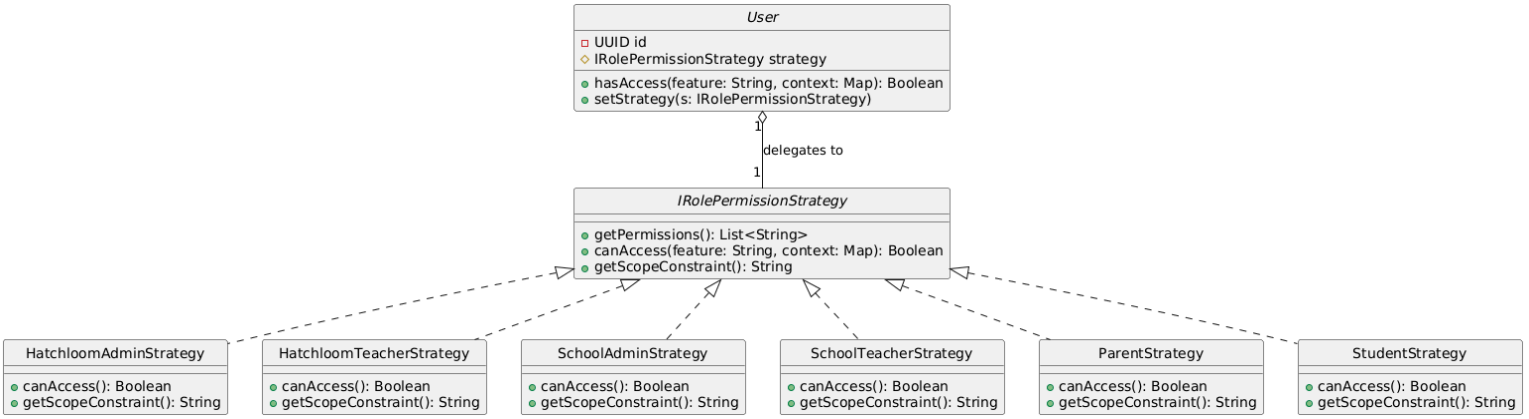
School	*UUID schoolId, String name*		*Client entity under Hatchloom that contains its own students, parents, and administrative staff.*
SchoolAdmin	*UUID schoolId*	*manageCohorts()*	



Class	*Attribute Name*	*Method Name*	*Description*
SessionManager (Singleton)	*instance: SessionManager, store: SessionStoreInterface,*	*getInstance(): SessionManager, createSession(user: User): Session, validateToken(token: String): User, invalidate(token: String): void,*	*Acts as the single point of truth for managing user sessions across the platform. It ensures only one instance exists to maintain global session consistency and security.*

Copyright Object Oriented Pty

SessionStore Interface (Interface)	*None,*	*save(session: Session): void, find(token: String): Session, delete(token: String): void,*	*Defines a contract for session persistence. This allows the system to swap storage backends such as a database or in-memory cache without changing the manager logic.*
Session	*token: String, expiresAt: DateTime, userId: UUID,*	*isExpired(): Boolean,*	*A data object representing an active user session. It contains the authentication token and expiration logic required to validate ongoing user activity.*



Class	*Attribute Name*	*Method Name*	*Description*

Copyright Object Oriented Pty

IRolePermission Strategy (Interface)	*None,*	*getPermissions(): List<String>, canAccess(feature: String, context: Map): Boolean, getScopeConstraint(): String*	*Defines the behavioral contract for all role-based permissions. It ensures every role implementation provides a list of allowed actions and a specific data scope such as Global vs. School-specific.*
User (Client)	*id: UUID, role: RoleType, strategy: IRolePermission Strategy,*	*setStrategy(s: IRolePermissionStrategy), hasAccess(feature: String, context: Map): Boolean*	*The primary context class that maintains user identity. It delegates all authorization logic to its assigned strategy object, allowing permissions to change dynamically based on the role.*
StudentStrategy	*None,*	*canAccess(): Boolean, getScopeConstraint(): String,*	*Implements permissions for the Primary User. Grants access to LaunchPad and ConnectHub while restricting data scope to their own records.*

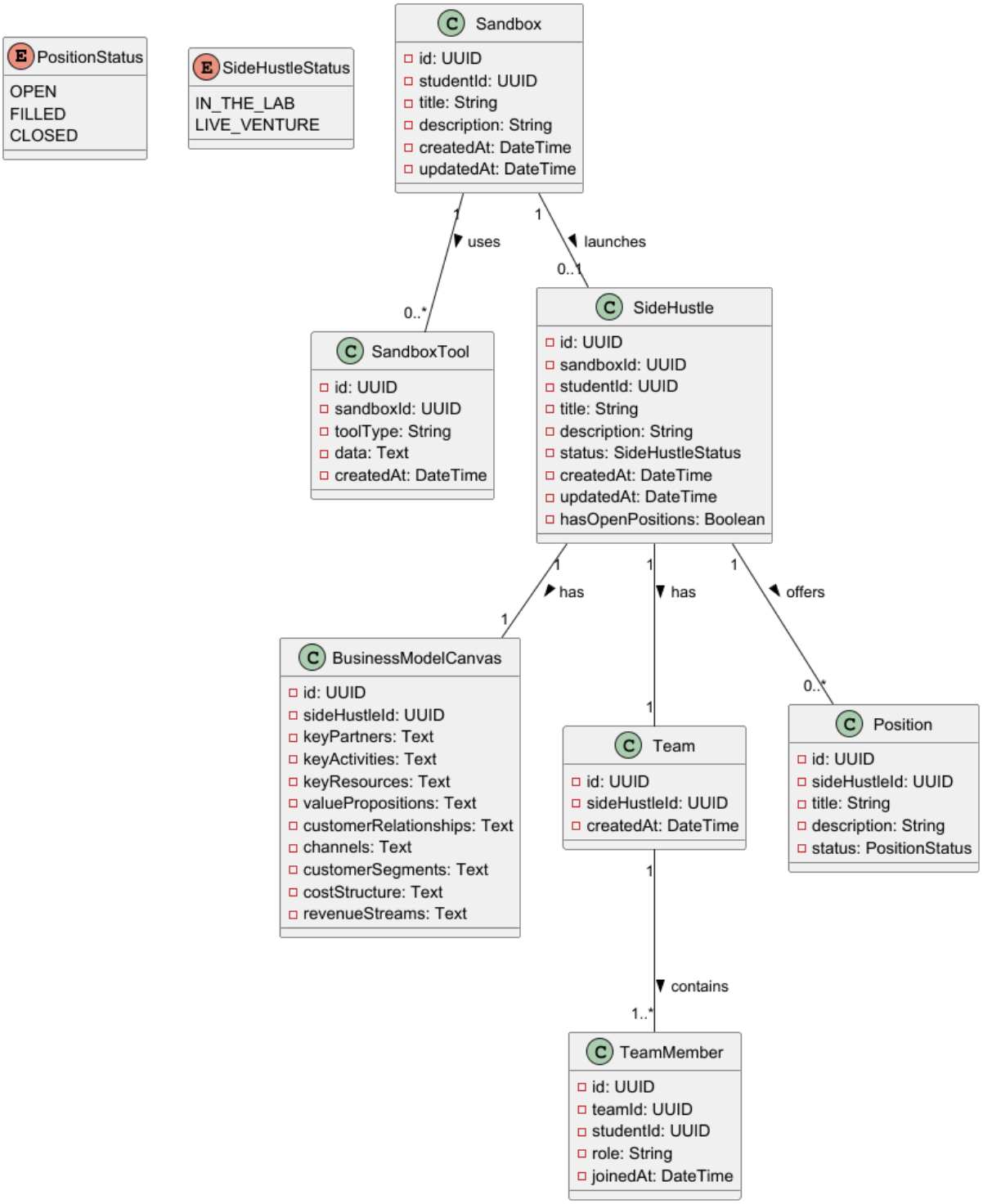
Copyright Object Oriented Pty

HatchloomTeacherStrategy	*None,*	*canAccess(), getScopeConstraint()*	*Implements permissions for Hatchloom employees. Grants Moderate and MonitorProgress capabilities with a CROSS_SCHOOL_GLOBAL scope.*
ParentStrategy	*None,*	*canAccess(): Boolean, getScopeConstraint(): String,*	*Implements read-only permissions for guardians. Access is strictly limited to the ReadChildWork feature for their specifically linked student.*
SchoolAdminStrategy	*None,*	*canAccess(): Boolean, getScopeConstraint(): String,*	*Implements administrative permissions scoped to a single institution. Grants access to CohortAnalytics within the SINGLE_SCHOOL_LIMIT constraint.*
EntrepreneurStrategy	*None,*	*canAccess(): Boolean, getScopeConstraint(): String,*	*Implements guest-level permissions. Restricts access primarily to PostChallenge and JoinCalls within a*

Copyright Object Oriented Pty

			highly restricted guest scope.
HatchloomAdminStrategy	*None,*	*canAccess(), getScopeConstraint()*	*Implements full system-wide permissions. Grants true for all features and UNRESTRICTED scope for school onboarding and system moderation.*
SchoolAdminStrategy	*None*	*canAccess(), getScopeConstraint()*	*Grants 'CohortAnalytics' validated against the user's specific organization.*

Copyright Object Oriented Pty



Class Name	*Attribute / Method*	*Description*
Sandbox	*id: UUID* *studentId: UUID* *title: String*	*Unique identifier for the sandbox project* *References the student who owns the sandbox*

Copyright Object Oriented Pty

	description: String *createdAt: DateTime* *updatedAt: DateTime*	*Name of the sandbox project* *Optional description of the sandbox* *Timestamp when the sandbox was created* *Timestamp when the sandbox was last updated*
SandboxTool	*id: UUID* *sandboxId: UUID* *toolType: String* *data: Text* *createdAt: DateTime*	*Unique identifier for the sandbox tool* *References the sandbox that owns this tool* *Type of tool (e.g., template, resource, helper)* *Tool data or content stored as text* *Timestamp when the tool was created*
SideHustle	*id: UUID* *sandboxId: UUID* *studentId: UUID* *title: String* *description: String* *status: SideHustleStatus (IN_THE_LAB / LIVE_VENTURE)* *createdAt: DateTime* *updatedAt: DateTime*	*Unique identifier for the side hustle* *Optional reference to the sandbox it launched from* *References the student who owns the side hustle* *Name of the side hustle* *Optional description of the side hustle* *Current stage of the side hustle*

Copyright Object Oriented Pty

	hasOpenPositions: Boolean	*Timestamp when side hustle was created* *Timestamp when side hustle was last updated* *True if any associated position has status OPEN*
BusinessModelCanvas	*id: UUID* *sideHustleId: UUID* *keyPartners: Text* *keyActivities: Text* *keyResources: Text* *valuePropositions: Text* *customerRelationships: Text* *channels: Text* *customerSegments: Text* *costStructure: Text* *revenueStreams: Text*	*Unique identifier for the BMC* *References the side hustle this BMC belongs to* *Key partners for the business* *Key activities of the business* *Key resources of the business* *Value propositions offered* *How the business manages customer relationships* *Distribution channels* *Target customer segments* *Costs associated with the business* *Revenue streams of the business*
Team	*id: UUID* *sideHustleId: UUID* *createdAt: DateTime*	*Unique identifier for the team* *References the side hustle this team belongs to*

Copyright Object Oriented Pty

		Timestamp when the team was created
TeamMember	*id: UUID* *teamId: UUID* *studentId: UUID* *role: String* *joinedAt: DateTime*	*Unique identifier for the team member* *References the team this member belongs to* *References the student assigned as a member* *Role of the team member (e.g., developer, designer)* *Timestamp when the member joined the team*
Position	*id: UUID* *sideHustleId: UUID* *title: String* *description: Text* *status: PositionStatus (OPEN / FILLED / CLOSED)*	*Unique identifier for the position* *References the side hustle this position belongs to* *Name of the role or position* *Description of responsibilities for this position* *Current status of the position*

6 Use of Design Patterns

In this section you will:

1. *List each design pattern used*
2. *Provide the rationale for each design pattern you used*
3. *Point where in your code the design pattern is implemented*

Factory design patterns and observer design patterns are used for the ConnectHub service. The reason the factory design pattern was used for the creation of different post types is because it allows the program to decide during runtime which class/post type to create based on the type that will be submitted to the Factory. This allows it so if new kinds of post

Copyright Object Oriented Pty

types wanted to be added, a lot of the code isn't placed within a specific class where adding a new class would require changing the entire code, but using Factory classes, we can override it in a subclass and change what kind of post type being created, and can be done since it has a common abstract class of Post.

For observer, the observer design pattern allows us to add subscriptions to the PostFeed where objects can subscribe or unsubscribe from events from that PostFeed class, especially when a new post is created. Here, whenever an important event happens like a new post is created, the observer pattern will loop over its subscribers and call the update method on the observer which will send the notifications, and will update the user feed with the new post, and it is loosely coupled since calling the update method does not need to know what happens next.

The Strategy design pattern is used to manage the complex, role-based permission logic for the six distinct user types (Student, Teacher, Parent, School Admin, Entrepreneur, and Platform Admin). By encapsulating the authorization logic into separate Strategy classes (e.g., TeacherStrategy, ParentStrategy), the system avoids massive, nested if-else or switch statements within the User class. This allows the platform to be "Open/Closed" new roles can be added by simply creating a new strategy class without modifying existing authentication code. It specifically handles unique constraints, such as the Teacher's cross-school access versus the School Admin's single-institution restriction. It is implemented via the IRolePermissionStrategy interface and its concrete implementations. The User class maintains a reference to this interface and delegates the hasAccess() call to the currently assigned strategy object.

The Singleton design pattern is applied to the SessionManager class to ensure that only one instance of the session coordinator exists during the application's lifecycle. Since session management involves a single point of truth for validating tokens and tracking active users, having multiple instances could lead to synchronization errors or security vulnerabilities. The Singleton ensures that every part of the system from the login controller to the middleware interacts with the same session store and global state. It is implemented in the SessionManager class through a private static instance attribute and a public getInstance() method, which serves as the global access point for session operations like createSession() and validateToken().

7 Activities Plan

7.1 Gantt Chart

In this section you will provide a revised Gantt Chart of your activities. You can use as a base the one submitted in Deliverable 1.

Team Quebec - GANTT Diagram.xlsx

1.1 Project Backlog and Sprint Backlog

In this Section, and assuming you follow a Scrum process model, provide a revised list (with respect to Deliverable 1) of product backlog items so that you can select items for your Sprint backlog. Make sure the product backlog list and the tasks in each product backlog item are consistent with the Gantt Chart in Section 7.1. above.

Project Backlog

Auth & User Profile Service

Copyright Object Oriented Pty

User Management

- Registering new users
- Linking parent-child accounts
- Storing passwords

User Story: As a new User, I want to be able to register a new account with a role so that I can access the system.

Authentication

- Logins with usernames and passwords
- Credential Validation

User Story: As a User, I want to be able to log in using my credentials

Session Management

- Creating session tokens on login
- Validating session tokens on every API call
- Expire sessions after timeouts
- Delete invalid/expired sessions

User Story: As the System, I want to validate session tokens on every API call so that only authenticated users can access certain resources.

Role Permissions

- Store user roles
- Enforce permissions based on the user's role
- Return user roles and permissions with session validation

User Story: As the System, I want to be able to enforce role-based permissions so that users can only access authorized features.

Profile CRUD

- Get user profiles
- Updating user profiles
- Listing all profiles (admin only)

User Story: As a User, I want to view and update my profile so that my information remains accurate

User Story: As an Admin, I want to be able to list all user profiles

LaunchPad Service

Sandbox Workspace

- Create Sandbox
- Get Sandbox
- Simplified Tools within the workspace
- Listing all Sandboxes for a student

User Story: As a Student, I want to be able to create and access sandbox workspaces so that I can experiment with ideas and manage my projects.

SideHustle

- Create SideHustle
- Get SideHustle

Copyright Object Oriented Pty

- Update SideHustle
- List all SideHustles for a student

User Story: As a student, I want to be able to create and manage my SideHustles so that I can develop and manage my potential ventures.

Business Model Canvas (BMC)

- Storing 9 standard sections
- Allow students to edit and save each section

User Story: As a student, I want to be able to save and edit any part of my BMC so I can clearly define my business strategy.

Team Management

- Create teams per SideHustle
- Adding team members
- Listing all team members
- Adding positions to team members

User Story: As a team leader, I want to be able to manage my team members and assign roles so that collaboration is organized and efficient.

Position Management

- Create position
- Update positions
- Flag SideHustle teams with open positions

User Story: As a SideHustle owner, I want to have the ability create and manage open positions so that I can recruit collaborators for my venture.

LaunchPad Home Aggregation

- List all Sandboxes for a student
- List all SideHustles for a student
- Return summary counts in Lab count and Live Ventures count

User Story: As a student, I want to be able to see an overview of all my Sandboxes and SideHustles.

ConnectHub Service

Feed System

- Create feed posts
- Share Achievements
- Retrieve Personalized Feeds
- Like posts
- Comment on posts
- Notify observers

User Story: As a student, I want to be able to share updates and engage with others through posts, likes, and comments.

Classifieds

- Create classified posts
- List classifieds
- Link classified posts to SideHustle positions
- Add statuses to each post (OPEN, FILLED, CLOSED)

Copyright Object Oriented Pty

- Update statuses
- Retrieve posts through ID

User Story: As a student, I want to be able to post and browse classified posts linked to SideHustles so that I can find or offer collaboration opportunities.

Messaging

- Create threads
- Send messages
- Associate messages with context

User Story: As a user, I want to be able to communicate with message threads linked to specific activities so that collaboration remains organized.

Sprint 1 Backlog

Sprint 1 Goal

Complete core system design for Auth, LaunchPad, and ConnectHub services.

Auth & User Profile Service

- Sequence diagram: user registers, logs in, and is directed to the dashboard
- Sequence diagram: session validation on an API call (middleware pattern)
- Class diagram for User, UserProfile, Session, Role
- Design pattern identification: potentially Singleton (session manager), Strategy (role-based permissions)
- 4 test cases (e.g., register user, login with valid/invalid credentials, session validation, role check)

LaunchPad Service

- Sequence diagram: student navigates to LaunchPad, opens SideHustle, edits BMC
- Sequence diagram: student posts an open position from SideHustle
- Class diagram for Sandbox, SideHustle, BMC, Team, Position
- 4 test cases (e.g., create SideHustle, update BMC, create open position, LaunchPad summary counts)

ConnectHub Service

- Sequence diagram: student shares win in ConnectHub feed, posts OPEN position in classifieds
- Class diagram for FeedItem, FeedAction, ClassifiedPost, Message, Thread
- Design pattern identification: Observer pattern for feed updates, or Factory for different post types
- 4 test cases (e.g., create feed post, list classifieds, create classified from position, filter classifieds by status)

1.2 Group Meeting Logs

In this Section you add the minutes of each meeting, listing the attendance, what the topics of discussion in the meeting were, any decisions that were made, and which team members were assigned which tasks. These minutes must be submitted with the project report in each deliverable and will provide input to be used for the overall assessment of the project.

Present Group Members	Meeting Date	Issues Discussed / Resolved

Copyright Object Oriented Pty

Andrew, Ronald, Jinyang, Anthony	Feb 28, 2026	UML Diagrams for each Subpack for the Hatchloom Project
Andrew, Ronald, Jinyang, Anthony	March 1st, 2026	Spring Backlog, Launchpad service

2 Test Driven Development

In this section you will extend/revise the test cases you have submitted in Deliverable 1. The idea is by Deliverable 3 you will have a comprehensive list of functional-level test cases. The information will be provided in the form of a table as follows:

Test ID	The unique Id of the test case
Category	Which part of the system is tested (*e.g. evaluation of user credentials stored on file or DB*)
Requirements Coverage	The unique ID of the requirement tested (*e.g. UC1-Successful-User-Login*)
Initial Condition	Initial conditions required for the test case to run (*e.g. the system has been initiated and runs*)
Procedure	The list of steps required for this test case (*e.g.* *1. The user selects login* *2. The user provides a user name* *3. The user provides a password* *4. The user logs-in into the system and is presented with the main UI window*)
Expected Outcome	The expected outcome of the test case (*e.g. the login form closes, and the user is presented with the main UI window*)
Notes	Any other notes you may want to add for this test case, which are also reflected in the requirements specification (*e.g. the user should provide only alphanumeric user names and passwords without any special characters*)

Test ID	1
Category	Sending API requests from the client to the server to create a post Validating input details server-side Using the factory method to determine what kind of post is created Storing post in the database
Requirements Coverage	HL-Post-Created-Success
Initial Condition	The system has to be on and running The user must be logged in

Copyright Object Oriented Pty

Procedure	*1. The user goes to the feed page and clicks on the create post button* *2. The user fills out the form and clicks submit*
Expected Outcome	The expected outcome of the test case is that the post creation form will close, and be presented with a confirmation message and returned back to the feed page.
Notes	Any other notes you may want to add for this test case, which are also reflected in the requirements specification (*e.g. the user should provide only alphanumeric user names and passwords without any special characters*)

Test ID	2
Category	Sending API requests from the client to the server to create a post Validating input details server-side Using the factory method to determine what kind of post is created Storing post in the database
Requirements Coverage	HL-Classified-Post-Success
Initial Condition	The system is running The user must be logged in The user must have at least one active side hustle
Procedure	1. The user goes to the side hustles page 2. The user clicks on an active project they have and are presented with the details 3. The user clicks on the create classified post button 4. The user fills out the information, specifying which project and the status, and clicks the submit button
Expected Outcome	The expected outcome of the test case is that the classified post creation form should close, and the user should be presented with a confirmation message, and returned back to the project page.
Notes	Any other notes you may want to add for this test case, which are also reflected in the requirements specification (*e.g. the user should provide only alphanumeric user names and passwords without any special characters*)

Test ID	3
Category	Sending API requests from the client to the server to create a post

Copyright Object Oriented Pty

	Updating post status in database
Requirements Coverage	HL-Classified-Post-Update
Initial Condition	The system has to be on The user has to be signed in The user has at least one side hustle The user has at least one classified post created
Procedure	1. The user goes to the side hustles page 2. The user clicks on their specific side hustle/project 3. The user will be brought to the project page and will be shown a list of classified posts they have created 4. The user will click on one of the posts 5. The user will update its status to a new status and click the submit button
Expected Outcome	The expected outcome of the test case is that the classified post page will close, and the user will be brought back to the projects page with a confirmation message
Notes	Any other notes you may want to add for this test case, which are also reflected in the requirements specification (*e.g. the user should provide only alphanumeric user names and passwords without any special characters*)

Test ID	4
Category	User Registration and Composite Lifecycle Management. Storing User-Parent relationship in the database.
Requirements Coverage	HL-Parent-Registration-Success
Initial Condition	The system is running. A Student account must already exist in the database with a unique UUID.
Procedure	1. The user selects the Parent Registration option. 2. The user provides a unique email, username, and password. 3. The user enters the Student UUID of the child they wish to monitor. 4. The user clicks the Register button.
Expected Outcome	The system validates that the Student exists, creates the Parent account as a dependent part of that Student, and redirects the user to the Parent Dashboard.
Notes	Per the Composition rule, the registration must fail if the Student ID is invalid. The Parent record is logically tied to the Student's existence.

Copyright Object Oriented Pty

Test ID	5
Category	Evaluation of user credentials stored in DB. Using Singleton SessionManager to generate a global session instance.
Requirements Coverage	HL-User-Login-Success
Initial Condition	The system is initiated and running. The user has a pre-existing account in the database.
Procedure	1. The user provides their registered email. 2. The user provides their password. 3. The user clicks the Login button. 4. The system calls SessionManager.getInstance() to initiate the session.
Expected Outcome	The login form closes, the user is authenticated, and they are presented with the main UI dashboard corresponding to their specific RoleType.
Notes	The password must be hashed and checked against the database; the email must be unique to the User identity class.

Test ID	6
Category	Session Management and Token Validation. Using SessionStoreInterface to find and delete session data.
Requirements Coverage	HL-Session-Invalidate-Logout
Initial Condition	The user is currently authenticated and has a valid, non-expired token.
Procedure	1. The user clicks on the Logout button in the navigation bar. 2. The SessionManager calls the invalidate method for the current token. 3. The user attempts to navigate back to the Dashboard using the browser's back button.
Expected Outcome	The session token is deleted from the store, and the user is redirected to the login page. Any attempt to access protected routes results in an "Unauthorized" error.

Copyright Object Oriented Pty

Notes	This ensures that the Singleton SessionManager correctly manages the lifecycle of the user's presence on the platform.

Test ID	7
Category	Session Management and Token Validation. Using SessionStoreInterface to find and delete session data.
Requirements Coverage	HL-Session-Invalidate-Logout
Initial Condition	The user is currently authenticated and has a valid, non-expired token.
Procedure	1. The user clicks on the Logout button in the navigation bar. 2. The SessionManager calls the invalidate method for the current token. 3. The user attempts to navigate back to the Dashboard using the browser's back button.
Expected Outcome	The session token is deleted from the store, and the user is redirected to the login page. Any attempt to access protected routes results in an "Unauthorized" error.
Notes	This ensures that the Singleton SessionManager correctly manages the lifecycle of the user's presence on the platform.

Test ID	8
Category	Evaluation of Role-Based Access Control (RBAC). Implementing Strategy Pattern logic to determine feature access.
Requirements Coverage	HL-RBAC-Teacher-Global-Access
Initial Condition	The system is running. A Teacher account exists. A Student account from a different school district exists.
Procedure	1. The Teacher logs into the system. 2. The Teacher navigates to the "Monitor Progress" section. 3. The Teacher searches for and selects a Student from a different school. 4. The system calls strategy.canAccess('MonitorProgress', context).

Copyright Object Oriented Pty

Expected Outcome	Access is granted. The Teacher is able to view the student's progress because the TeacherStrategy returns a CROSS_SCHOOL_GLOBAL scope constraint.
Notes	This verifies that the Teacher role correctly bypasses the single-school restriction that applies to the School Admin role.

Test ID	9
Category	Evaluation of Scoped Access for School Administrators. Using Strategy Pattern to enforce data boundaries.
Requirements Coverage	HL-RBAC-Admin-Scope-Restriction
Initial Condition	The system is running. A School Admin for "School A" is logged in.
Procedure	1. The School Admin attempts to access the "Cohort Analytics" page. 2. The School Admin attempts to filter data for "School B". 3. The system executes the SchoolAdminStrategy.getScopeConstraint() method.
Expected Outcome	The system displays analytics for "School A" but returns an "Access Denied" or empty result for "School B" data.
Notes	This validates the SINGLE_SCHOOL_LIMIT constraint defined in the strategy pattern to protect school-specific data privacy.

Test ID	10
Category	User Registration. Storing User-Role relationship in the database.
Requirements Coverage	HL-Student-Registration-Success
Initial Condition	The system is running. No existing account with the provided email or username exists in the database.
Procedure	1. The user navigates to the registration page and selects the Student registration option. 2. The user provides a unique email, username, password, and their school ID. 3. The user clicks the Register button.

Copyright Object Oriented Pty

	4. The system validates the input, hashes the password, and stores the new Student record in the database with the role set to “student”.
Expected Outcome	The student account is successfully created and stored in the database. The user is redirected to the Student Dashboard corresponding to their RoleType.
Notes	Per the system’s validation rules, usernames and passwords must be alphanumeric only and contain no special characters. Registration must fail if the provided school ID does not match an existing school in the database.

Test ID	11
Category	LaunchPad SideHustle creation. Using the Factory Method to instantiate the correct project type. Storing SideHustle in the database.
Requirements Coverage	HL-SideHustle-Created-Success
Initial Condition	The system is running. The user is logged in as a Student. The user has at least one active Sandbox.
Procedure	1. The user navigates to the LaunchPad page and opens an active Sandbox. 2. The user clicks the “Create SideHustle” button. 3. The user fills out the SideHustle creation form, providing a title, description, and project type. 4. The user clicks the Submit button.
Expected Outcome	The SideHustle is created and stored in the database, linked to the parent Sandbox. The creation form closes, and the user is returned to the LaunchPad overview with a confirmation message.
Notes	A SideHustle must be associated with an existing Sandbox; creation must fail if no valid Sandbox ID is provided. The Factory Method must determine and instantiate the correct SideHustle subtype based on the project type field.

Test ID	12
Category	Business Model Canvas (BMC) update via API. Sending API requests to update individual BMC section fields. Storing updated BMC data in the database.
Requirements Coverage	HL-BMC-Update-Success

Copyright Object Oriented Pty

Initial Condition	The system is running. The user is logged in. The user has at least one active SideHustle with an existing, initialized BMC.
Procedure	1. The user navigates to their SideHustle page and opens the BMC tool. 2. The user selects a BMC section (e.g., “Value Proposition”) and edits its content. 3. The user clicks the Save button for that section. 4. The system sends a PATCH request to the BMC API endpoint with the updated field data and SideHustle ID.
Expected Outcome	The updated BMC section is persisted to the database. The user is presented with a confirmation message and the BMC tool reflects the newly saved content.
Notes	Each BMC section is saved independently; a failed save on one section must not overwrite data in other sections. The BMC record must be linked to the correct SideHustle ID in the database.

Test ID	13
Category	Classified post state management. Updating position state (OPEN -> FILLED) in the database via the ConnectHub service.
Requirements Coverage	HL-Position-State-Filled-Success
Initial Condition	The system is running. The user is logged in. The user owns at least one Classified post currently in the OPEN state.
Procedure	1. The user navigates to their SideHustle project page. 2. The user views the list of Classified posts they have created. 3. The user selects an OPEN position post. 4. The user clicks “Mark as Filled” and confirms the action. 5. The system sends a PATCH request updating the post’s state to FILLED in the database.
Expected Outcome	The Classified post status is updated to FILLED in the database. The user is returned to the project page with a confirmation message, and the post now displays the FILLED state.
Notes	Only the owner of the Classified post may change its state; the system must return an “Unauthorized” error if another user attempts this action. A post in the FILLED state must

Copyright Object Oriented Pty

	not be modifiable back to OPEN without an explicit re-open action.

Copyright Object Oriented Pty